

Machine Learning Workflows and Evaluation

15 hr | Subject Code: 24CV201T | ODD SEM 2026-27

Dr. Ankit Deshmukh

2026-10-06

Contents

1	What You Will Learn in Unit 3	3
2	The Machine Learning Workflow	4
2.1	What is Machine Learning, really?	4
2.2	Walking through each step	4
3	Data Preparation – Cleaning the Site Before Building	5
3.1	Data Cleaning – Missing values, duplicates, impossible values	5
3.2	Normalization and Scaling – Putting all columns on one measuring tape	7
3.3	Encoding Categorical Variables – Turning words into numbers	8
4	Feature Engineering for Civil Data – Making Good Bricks from Raw Clay	9
4.1	Rainfall Intensity ($I = P/t$)	10
4.2	Runoff Coefficient (C) and the Rational Method	10
4.3	NDVI Anomalies – Satellite health check for vegetation	11
4.4	Why Engineered Features Matter – A quick preview	13
5	Model Training and Validation	14
5.1	What is training and validation?	14
5.2	Splitting the data – the mock exam and the final exam	15
5.3	Training a linear regression model	15
5.4	Predicting on the unseen test set	16
6	Evaluation Metrics for Regression (RMSE, MAE, R-squared)	17
6.1	What are we measuring?	17
6.2	Computing the metrics	17
6.3	Always compare with a lazy baseline	18

7	Evaluation Metrics for Classification (Accuracy, Precision, Recall, F1, ROC)	20
7.1	What is classification?	20
7.2	The flood-warning dataset (simulated for reproducibility)	20
7.3	The Confusion Matrix – Four verbs of judgement	20
7.4	The Four Key Metrics	21
7.5	The Threshold Slider – Same model, different personalities	22
7.6	The ROC Curve – Judging across ALL thresholds at once	23
8	Statistical Challenges – Overfitting, Underfitting and the Bias-Variance Tradeoff	25
8.1	What is the Goldilocks problem?	25
8.2	The experiment: seasonal groundwater level	25
8.3	The Bias-Variance Tradeoff	27
9	Cross-Validation Techniques – Many Mock Exams, Not One	28
9.1	What is cross-validation?	28
9.2	Cross-validation in practice	29
10	One-Page Summary	30
11	Glossary	31
12	Exit Ticket	31
13	Further Reading	32

1 What You Will Learn in Unit 3

This unit is the *engineering manual* for building a machine learning (ML) model. If Unit 1 gave you the vocabulary and Unit 2 gave you the toolbox, **Unit 3 gives you the construction sequence** – the ordered set of steps every competent engineer follows, whether they are predicting the strength of a concrete cube or the peak flow of a flash flood.

By the end of this handout, you will be able to:

1. **Describe the full Machine Learning workflow** – from asking the right question to deploying a monitored model. *(CO4)*
2. **Prepare civil engineering data properly**: clean it (missing values, duplicates, impossible readings), scale it (min–max and z-score), and encode categorical variables (label vs one-hot). *(CO3)*
3. **Engineer smart features** that carry physical meaning – rainfall intensity, weighted runoff coefficient, NDVI anomaly, and log-transformed curing age. *(CO3)*
4. **Train and validate models** using a disciplined train/test split. *(CO4)*
5. **Evaluate regression models** with MAE, RMSE, and R-squared – and know *which* metric to trust. *(CO5)*
6. **Evaluate classification models** with Accuracy, Precision, Recall, F1, and ROC-AUC – and know why accuracy alone can get people killed. *(CO5)*
7. **Diagnose overfitting vs. underfitting** using the bias–variance tradeoff, and use **k-fold cross-validation** to select the model that will actually work on new data. *(CO5)*

Civil connection: Every idea in this unit is illustrated with a civil engineering example – concrete strength, urban floods, rainfall, runoff, vegetation health, and groundwater. The tools are general; the problems are the ones you will meet on site.

How to use this handout: Read a section, then run its code chunk in RStudio (or knit the whole file). Change the numbers. Break the data on purpose. Watch what the model does. That is how engineers learn machines – by poking them until they behave.

How to read the outputs: Wherever a code chunk prints a table or a figure, the section immediately after it (marked with a purple bar like this one) explains what you are looking at, what it means, and what to watch out for.

2 The Machine Learning Workflow

2.1 What is Machine Learning, really?

Machine Learning is the practice of **teaching a computer to find patterns in data** so that it can make predictions on **new, unseen data**. The crucial phrase is *unseen*. A computer that has memorised every answer in the textbook has not learned anything – it has just stored information. A computer that can answer a question it has never seen has *learned*.

The workflow is the step-by-step recipe we always follow. It is not optional. Skipping a step is like skipping compaction before laying a road – the failure shows up later, and it is expensive.

Step	What happens	Civil engineering example
1. Ask a question	Define the prediction goal	“What will be the 28-day strength of this concrete mix?”
2. Collect data	Gather measurements	Cube-crushing test records: cement, water, age, strength
3. Prepare data	Clean, scale, encode, engineer features	Fix missing sensor readings, convert soil types to numbers
4. Split data	Separate into training and testing sets	Mock exam vs. final exam
5. Train the model	Let the algorithm learn patterns	Model learns “more cement + proper curing gives stronger”
6. Evaluate	Check accuracy on unseen test data	RMSE of strength predictions
7. Use and improve	Deploy, monitor, retrain	Live strength predictor on the batching plant

2.2 Walking through each step

Step 1 – Ask a question. A good ML project starts with a *specific* question, not a vague hope. “Can we predict concrete strength?” is vague. “Can we predict 28-day compressive strength within ± 3 MPa from mix proportions and curing age?” is a real engineering target. The question determines the data you need, the model you build, and the metric you use to judge success.

Step 2 – Collect data. Data comes from lab registers, sensor logs, satellite imagery, site inspection reports, and design drawings. Collect *more* than you think you need – models improve with data, but only up to a point.

Step 3 – Prepare data. This is the unglamorous 60–80% of the work. Real data is messy: sensor failures, double entries, physically impossible values, mixed units, and categorical text like “clay” or “gravel”. Algorithms cannot handle most of this. We fix it here.

Step 4 – Split data. We hold back a portion of the data (usually 20%) and *never let the model see it during training*. This is the honest exam at the end of the course.

Step 5 – Train the model. The algorithm adjusts its internal numbers (coefficients, splits, weights) to reduce error on the training set. This is where the “learning” happens.

Step 6 – Evaluate. We predict the held-out test set and compute metrics. This is the only number that tells the truth about how the model will behave in the field.

Step 7 – Use and improve. Deploy the model. Watch its performance. When it drifts (because materials change, climate shifts, or sensors age), retrain it. ML is a cycle, not a one-time build.

Key idea: The model is like a student. You teach it with the **training set** (mock exams) and judge it with the **test set** (final exam it has never seen). Judging a student on questions they already memorised proves nothing.

Never evaluate on training data. A model graded on data it has already seen will always look artificially good. This is called **data leakage**, and it is the number one beginner mistake. It is like letting a student sit the final exam with the answer key in their lap – the score is meaningless.

3 Data Preparation – Cleaning the Site Before Building

Civil connection: You never build on an uncleared site. You remove debris, level the ground, and set out the grid before the first column is cast. Data preparation is exactly this – and in practice it takes **60–80% of a data scientist’s time**. Skipping it is the leading cause of models that look great in the lab and fail in the field.

3.1 Data Cleaning – Missing values, duplicates, impossible values

Why does it matter? Real sensor logs and lab registers always have gaps (sensor failure during a storm), repeats (a technician typed the row twice), and errors (a curing age of -5 days is physically impossible). Most ML algorithms **cannot handle missing values at all** – they will either crash or silently produce garbage. Cleaning is not optional.

Let us build a small, deliberately messy concrete dataset – the kind of thing you would actually find in a site register after a busy week.

```
set.seed(123)
cement <- round(runif(12, 250, 450), 1)      # kg/m3
water  <- round(runif(12, 150, 220), 1)      # litres/m3
age     <- sample(c(7, 14, 28, 90), 12, replace = TRUE) # curing days
strength <- round(
  20 + 0.08 * cement - 0.10 * water + 0.15 * age +
  rnorm(12, 0, 3),
  1
) # MPa

concrete <- data.frame(cement, water, age, strength)

# --- Inject real-world problems, like a messy lab register ---
concrete$cement[3] <- NA      # sensor failed during batching
concrete$water[6] <- NA      # reading lost
concrete$strength[9] <- NA    # cube crushed incorrectly
concrete$age[4] <- -5         # impossible: curing age can't be negative
concrete <- rbind(concrete, concrete[2, ]) # same row typed twice

concrete
```

```
##      cement water age strength
## 1    307.5 197.4   7     28.0
## 2    407.7 190.1  28     36.4
## 3         NA 157.2  90     41.1
```

```
## 4  426.6 213.0 -5    34.3
## 5  438.1 167.2 28    39.4
## 6  259.1    NA 14    25.4
## 7  355.6 173.0 7     30.3
## 8  428.5 216.8 14    29.6
## 9  360.3 212.3 28     NA
## 10 341.3 198.5 90    41.4
## 11 441.4 194.8 14    34.5
## 12 340.7 219.6 7     30.1
## 21 407.7 190.1 28    36.4
```

What you are seeing: A 13-row data frame (12 unique + 1 duplicate) with four columns: `cement`, `water`, `age`, and `strength`. Look carefully and you will spot:

- Row 3 has NA for cement (sensor failed).
- Row 6 has NA for water (reading lost).
- Row 9 has NA for strength (cube mis-tested).
- Row 4 has `age = -5` – physically impossible.
- Row 13 is an exact copy of row 2 – a duplicate.

This is what “raw data” looks like. It is normal. What is *not* normal is feeding it straight into a model.

Now the cleaning steps – count the damage, remove duplicates, fix impossible values, then handle gaps:

```
# 1) How many missing values per column?
colSums(is.na(concrete))
```

```
## cement    water    age strength
##      1         1      0         1
```

```
# 2) Remove duplicate rows
```

```
concrete <- concrete[!duplicated(concrete), ]
```

```
# 3) Impossible value -> convert to NA (we don't trust it)
```

```
concrete$age[concrete$age < 0] <- NA
```

```
# 4) Strategy A: DROP incomplete rows (simple, loses data)
```

```
concrete_dropped <- na.omit(concrete)
```

```
# 5) Strategy B: IMPUTE - fill gaps with the median (keeps all rows)
```

```
concrete_imputed <- concrete
```

```
concrete_imputed$cement[is.na(concrete_imputed$cement)] <-
  median(concrete_imputed$cement, na.rm = TRUE)
```

```
concrete_imputed$water[is.na(concrete_imputed$water)] <-
  median(concrete_imputed$water, na.rm = TRUE)
```

```
concrete_imputed$strength[is.na(concrete_imputed$strength)] <-
  median(concrete_imputed$strength, na.rm = TRUE)
```

```
concrete_imputed$age[is.na(concrete_imputed$age)] <-
  median(concrete_imputed$age, na.rm = TRUE)
```

```
c(
  "Before cleaning" = 13,
  "After dropping NA" = nrow(concrete_dropped),
  "After imputing NA" = nrow(concrete_imputed)
)
```

```
## Before cleaning After dropping NA After imputing NA
##                13                8                12
```

```
head(concrete_imputed)
```

```
## cement water age strength
## 1  307.5 197.4  7    28.0
## 2  407.7 190.1 28    36.4
## 3  360.3 157.2 90    41.1
## 4  426.6 213.0 14    34.3
## 5  438.1 167.2 28    39.4
## 6  259.1 197.4 14    25.4
```

How to read this output:

- `colSums(is.na(concrete))` printed the count of missing values in each column. If you saw `cement = 1`, `water = 1`, `age = 0`, `strength = 1`, that confirms three gaps.
- The final vector shows the trade-off: dropping leaves us with **fewer rows** (we lost every row that had any missing value), while imputing keeps **all unique rows**.
- The `head()` of the imputed data shows the median values sitting in the gaps – they look plausible, but remember they are *guesses*, not measurements.

Drop vs. impute?

- **Drop** when only a few rows are missing (say less than 5%) and you have plenty of data.
- **Impute** when data is precious – e.g., expensive field tests, rare flood events, or long-term monitoring.
- Use the **median** rather than the mean for skewed or outlier-prone columns, because one wild value can drag the mean far away while the median barely moves.

3.2 Normalization and Scaling – Putting all columns on one measuring tape

Why does it matter? In our concrete data, cement ranges roughly 200–500 while water ranges roughly 140–230. A distance-based algorithm (KNN, K-means – coming in Unit 4) would think cement is “much more important” simply because its numbers are bigger. Scaling puts every feature on a fair, comparable range so the algorithm judges importance by *pattern*, not by *unit*.

Two standard methods:

- **Min-Max Normalization** – squeezes everything into the range 0 to 1:

$$x' = \frac{x - x_{min}}{x_{max} - x_{min}}$$

- **Z-score Standardization** – centres the data at 0 with a spread (standard deviation) of 1:

$$z = \frac{x - \mu}{\sigma}$$

```
min_max <- function(x) (x - min(x)) / (max(x) - min(x))

concrete_imputed$cement_minmax <- round(min_max(concrete_imputed$cement), 3)
concrete_imputed$water_minmax <- round(min_max(concrete_imputed$water), 3)
concrete_imputed$strength_z <- round(as.numeric(scale(concrete_imputed$strength)), 3)

head(concrete_imputed[, c("cement", "cement_minmax",
                          "water", "water_minmax",
                          "strength", "strength_z")])
```

```
##   cement cement_minmax water water_minmax strength strength_z
## 1  307.5         0.265 197.4         0.644      28.0      -1.102
## 2  407.7         0.815 190.1         0.527      36.4       0.512
## 3  360.3         0.555 157.2         0.000      41.1       1.416
## 4  426.6         0.919 213.0         0.894      34.3       0.109
## 5  438.1         0.982 167.2         0.160      39.4       1.089
## 6  259.1         0.000 197.4         0.644      25.4      -1.601
```

```
# Cement naturally "shouts louder" because its range is wider:
round(range(concrete_imputed$cement), 0) # cement spread ~200 units
```

```
## [1] 259 441
```

```
round(range(concrete_imputed$water), 0) # water spread ~70 units
```

```
## [1] 157 220
```

How to read this output:

- The first table shows the original columns next to their scaled versions. For example, a cement value near the minimum becomes close to 0; one near the maximum becomes close to 1.
- **strength_z** is a z-score: +1.2 means “1.2 standard deviations above the site average strength”. -0.8 means “0.8 standard deviations below”.
- The second pair of lines shows the raw ranges: cement spans about 200 units, water about 70. Without scaling, a distance metric would let cement dominate water by roughly 3 times purely because of its units.

How to interpret a z-score: a z-score of +2 means “2 standard deviations above the mean of this dataset”. Because the units are removed, z-scores are instantly comparable across any quantity – MPa, GPa, mm/hr, or NDVI. This is why z-scores are the standard input for many statistical models and neural networks.

Think about it: Comparing compressive strength (~30 MPa) with elastic modulus (~30,000 MPa) without scaling is like measuring a column in millimetres and its shadow in metres – the numbers mislead you even though both lengths are correct in their own units.

3.3 Encoding Categorical Variables – Turning words into numbers

Why does it matter? Algorithms do mathematics. They cannot multiply “clay” by 2 or add “gravel” to “sand”. Soil types, road surface types, material grades, and land-use classes must all become numbers before a model can use them.

There are two common strategies:

```
soil <- c(
  "clay", "sand", "gravel", "clay", "sand", "gravel",
  "clay", "clay", "sand", "gravel", "sand", "clay"
)

# Option 1: LABEL encoding - factor() stores categories as integer codes
soil_factor <- factor(soil)
soil_factor
```

```
## [1] clay sand gravel clay sand gravel clay clay sand gravel
## [11] sand clay
## Levels: clay gravel sand
```

```
# Option 2: ONE-HOT encoding - a separate 1/0 column per category
onehot <- model.matrix(~ 0 + soil_factor)
colnames(onehot) <- levels(soil_factor)
head(onehot)
```

```
##   clay gravel sand
## 1    1     0    0
## 2    0     0    1
## 3    0     1    0
## 4    1     0    0
## 5    0     0    1
## 6    0     1    0
```

How to read this output:

- The `soil_factor` line prints the categories with their **levels** listed underneath. Internally, R stores “clay” as 1, “gravel” as 2, “sand” as 3 (alphabetical order by default).
- The one-hot matrix has **one column per category**. For a row that is “clay”, the clay column = 1 and gravel/sand = 0. Every row contains exactly one 1 and two 0s. That is why it is called “one-hot”.
- In the printed head, you should see something like: `clay gravel sand` across the top, and three rows of 0/1 beneath.

Label encoding can mislead. If clay = 1, gravel = 2, sand = 3, the model may wrongly infer that `sand > gravel > clay` – an ordering that does not exist. For **nominal** categories (no natural order), always prefer **one-hot encoding**. Reserve label encoding for **ordinal** categories – like `low < medium < high` bearing capacity – where the order is meaningful.

4 Feature Engineering for Civil Data – Making Good Bricks from Raw Clay

What is it? Feature engineering means **creating new, more meaningful columns** from raw measurements using engineering formulas. A good site engineer never works with raw numbers alone – they compute derived quantities like stress, unit weight, or discharge. A good ML engineer does the same. Engineered features are what turn a mediocre model into a useful one.

4.1 Rainfall Intensity ($I = P/t$)

Why depth alone lies. A 66 mm storm over 6 hours is a gentle soaking rain. A 30 mm storm in 1 hour is a downpour that will flood a culvert. Drainage design cares about **intensity**, not just depth:

$$I = \frac{\text{Rainfall depth } P \text{ (mm)}}{\text{Duration } t \text{ (hr)}} \quad (\text{mm/hr})$$

```
storms <- data.frame(
  storm_id   = paste0("Storm ", 1:6),
  depth_mm   = c(15, 42, 30, 66, 8, 55),
  duration_h = c(2, 6, 1, 3, 0.5, 5)
)
storms$intensity_mmh <- storms$depth_mm / storms$duration_h

storms[order(-storms$intensity_mmh), ]
```

```
##   storm_id depth_mm duration_h intensity_mmh
## 3   Storm 3      30         1.0          30.0
## 4   Storm 4      66         3.0          22.0
## 5   Storm 5       8         0.5          16.0
## 6   Storm 6      55         5.0          11.0
## 1   Storm 1      15         2.0           7.5
## 2   Storm 2      42         6.0           7.0
```

How to read this output:

- The table is sorted from most intense to least intense storm (the minus sign in `order(-...)` reverses the sort).
- Storm 4 has the **biggest depth** (66 mm). Storm 3 has the **highest intensity** (30 mm/hr) despite being only the third-deepest storm.
- Storm 5 is tiny in depth (8 mm) but in just 0.5 hours it reaches 16 mm/hr – halfway between the biggest and smallest here.

Look carefully: A model fed only “depth” would rate Storm 4 as the most dangerous. But a drainage engineer knows Storm 3 is the real problem – the culvert cannot carry 30 mm/hr, no matter that the total was only 30 mm. **This single engineered feature changes the entire ranking of risk.**

4.2 Runoff Coefficient (C) and the Rational Method

What is it? The runoff coefficient C tells what fraction of falling rain becomes surface flow. The rest infiltrates into the soil, evaporates, or is intercepted by vegetation. Typical values:

Surface	Typical C
Roof / pavement	0.85–0.95
Gravel	0.40–0.60
Lawn / grass	0.10–0.30
Forest	0.05–0.25

Rational Method (the practical form used in design offices):

$$Q \text{ (m}^3\text{/s)} = \frac{C \times i \text{ (mm/hr)} \times A \text{ (ha)}}{360}$$

```

C_paved    <- 0.90
C_grass    <- 0.20
area_paved <- 15
area_grass <- 10      # hectares, total catchment = 25 ha
rain_i     <- 50      # design storm intensity, mm/hr

C_weighted <- (C_paved * area_paved + C_grass * area_grass) /
              (area_paved + area_grass)
Q_current  <- C_weighted * rain_i * 25 / 360
Q_allpaved <- 0.90 * rain_i * 25 / 360 # future scenario: all concreted

c(
  C_weighted = round(C_weighted, 2),
  Q_now_m3s  = round(Q_current, 2),
  Q_future_m3s = round(Q_allpaved, 2)
)

```

```

##   C_weighted   Q_now_m3s Q_future_m3s
##         0.62         2.15         3.12

```

How to read this output:

- **C_weighted** is the area-weighted runoff coefficient of the current mixed catchment (part paved, part grass). It lands between 0.20 and 0.90, closer to the paved value because pavement covers 60% of the area.
- **Q_now_m3s** is today's peak discharge for a 50 mm/hr storm over 25 ha.
- **Q_future_m3s** is the future scenario where all gardens are paved. The value jumps dramatically – roughly a 45% increase in peak flow for the same rainfall.

Civil connection: This single engineered feature lets a model understand **urbanisation**, not just weather. Without it, two storms with identical rainfall would produce identical predictions – even if one falls on a forest and the other on a car park. The feature encodes what every drainage engineer knows instinctively.

4.3 NDVI Anomalies – Satellite health check for vegetation

What is it? The **Normalized Difference Vegetation Index (NDVI)** uses satellite reflectance to measure how green (i.e., how healthy) land is:

$$NDVI = \frac{NIR - Red}{NIR + Red}$$

Healthy leaves reflect a lot of Near-Infrared (NIR) light and absorb Red light, so NDVI approaches 1. Bare soil, water, and urban surfaces give NDVI near 0. An **anomaly** = this month's NDVI minus the normal NDVI – a red flag for drought, deforestation, or irrigation failure.

```

ndvi_data <- data.frame(
  month = factor(month.abb, levels = month.abb),
  red = c(0.20, 0.22, 0.19, 0.15, 0.12, 0.10,
           0.09, 0.10, 0.13, 0.16, 0.18, 0.21),
  nir = c(0.25, 0.28, 0.32, 0.38, 0.45, 0.50,
           0.52, 0.50, 0.44, 0.36, 0.30, 0.26)
)
ndvi_data$ndvi <- (ndvi_data$nir - ndvi_data$red) /
  (ndvi_data$nir + ndvi_data$red)

# "Normal" here = yearly mean (for teaching). In practice, compare each
# month against the 10-year average for that SAME month.
normal <- mean(ndvi_data$ndvi)
ndvi_data$anomaly <- round(ndvi_data$ndvi - normal, 3)

ndvi_data[, c("month", "ndvi", "anomaly")]

```

```

##      month      ndvi anomaly
## 1   Jan 0.1111111 -0.291
## 2   Feb 0.1200000 -0.282
## 3   Mar 0.2549020 -0.147
## 4   Apr 0.4339623  0.032
## 5   May 0.5789474  0.177
## 6   Jun 0.6666667  0.265
## 7   Jul 0.7049180  0.303
## 8   Aug 0.6666667  0.265
## 9   Sep 0.5438596  0.142
## 10  Oct 0.3846154 -0.017
## 11  Nov 0.2500000 -0.152
## 12  Dec 0.1063830 -0.295

```

How to read this output:

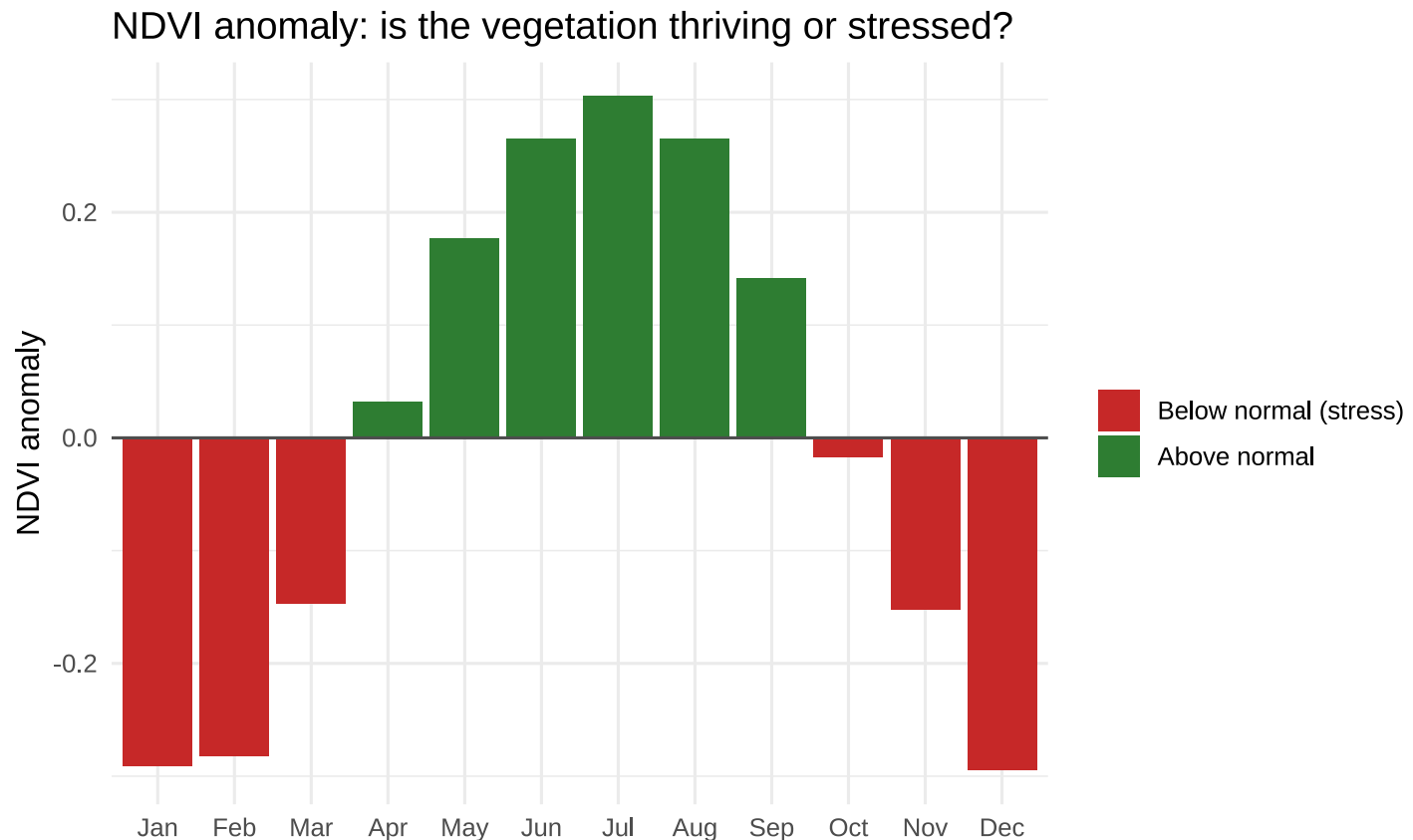
- Each row is one month. **ndvi** is the vegetation index for that month; **anomaly** is how far it deviates from the yearly average.
- **Positive anomaly** (green months) = vegetation is greener than average.
- **Negative anomaly** (dry months) = vegetation is stressed, browning, or missing.
- You will typically see positive anomalies in the monsoon months and negative anomalies in the pre-monsoon summer.

```

ggplot(ndvi_data, aes(x = month, y = anomaly, fill = anomaly > 0)) +
  geom_col() +
  scale_fill_manual(
    values = c("TRUE" = "#2e7d32", "FALSE" = "#c62828"),
    labels = c("FALSE" = "Below normal (stress)",
               "TRUE" = "Above normal"),
    name = NULL
  ) +
  geom_hline(yintercept = 0, color = "grey30") +
  labs(
    title = "NDVI anomaly: is the vegetation thriving or stressed?",
    x = NULL, y = "NDVI anomaly"
  )

```

```
) +  
theme_minimal(base_size = 12)
```



How to read this figure:

- The horizontal grey line at $y = 0$ is “normal”. Bars above it are green months, bars below it are stressed months.
- Tall green bars in the monsoon (Jun–Sep) mean the vegetation responded strongly to the rain – a healthy catchment.
- Red bars extending far below zero in the summer indicate drought stress, which in turn predicts lower soil moisture, weaker root cohesion, and a higher landslide risk in the following monsoon.
- **What a model can learn:** the pattern of anomalies over months is a fingerprint of the local climate-vegetation system. Any deviation from that fingerprint (an unusually red monsoon, say) can be flagged automatically by an ML model as an anomaly worth investigating.

Civil connection: NDVI anomalies feed directly into landslide risk (dead roots = weak slopes), irrigation planning, reservoir sedimentation studies, and urban heat-island mapping – the exact kinds of spatial applications you will meet in Unit 4.

4.4 Why Engineered Features Matter – A quick preview

```
# We build this dataset properly in Section 5; the message is simple:
cor(strength, age)      # raw curing age: weaker, confusing signal
cor(strength, log_age)  # engineered log(age): much clearer signal
```

Concrete gains strength **fast in the early days, slowly later** – a logarithmic pattern. Giving the model `log(age)` instead of raw `age` is like handing a student a neatly organised formula sheet instead of a pile of raw notes. The underlying information is the same, but the *shape* of the information is far easier to learn from.

We will prove this numerically in the next section.

5 Model Training and Validation

5.1 What is training and validation?

Training = the algorithm adjusts itself to fit the **training set**. It looks at thousands of examples and tunes its internal parameters to minimise error.

Validation = checking how well it does on the **unseen test set**. This is the moment of truth – the model has never seen these rows.

We use an **80/20 split** by convention: 80% for learning, 20% for honest examination. Some practitioners use 70/30 or 90/10; the right ratio depends on how much data you have. With very little data, cross-validation (Section 9) is safer.

```
set.seed(42)
n <- 120
concrete_ml <- data.frame(
  cement = runif(n, 200, 500),          # kg/m3
  water  = runif(n, 140, 230),          # litres/m3
  age    = sample(c(3, 7, 14, 28, 90), n, replace = TRUE) # days
)

# --- Feature engineering (Section 4 in action!) ---
concrete_ml$log_age <- log10(concrete_ml$age)

concrete_ml$strength <- round(
  5 + 0.08 * concrete_ml$cement - 0.10 * concrete_ml$water +
  12 * concrete_ml$log_age + rnorm(n, 0, 2.5),
  1
) # MPa

head(concrete_ml)
```

```
##      cement    water age  log_age strength
## 1 474.4418 172.1050   3 0.4771213      31.0
## 2 481.1226 176.9572  14 1.1461280      36.5
## 3 285.8419 191.6128  14 1.1461280      27.6
## 4 449.1343 193.0710  28 1.4471580      39.3
## 5 392.5237 204.7692  28 1.4471580      33.1
## 6 355.7288 175.5476  28 1.4471580      34.5
```

How to read this output: Each row is one concrete cube. `cement`, `water`, and `age` are the inputs; `log_age` is the engineered feature; `strength` is the target we want to predict. Notice that `log_age` is not in the same units as `age` – it is a dimensionless transformed variable.

```
c(
  "Correlation with raw age" = round(cor(concrete_ml$strength, concrete_ml$age), 3),
  "Correlation with log(age)" = round(cor(concrete_ml$strength, concrete_ml$log_age), 3)
)

## Correlation with raw age Correlation with log(age)
##                0.507                0.589
```

How to read this output: The two numbers are Pearson correlations between strength and each version of “age”. The `log_age` correlation should be **noticeably higher** (closer to +1). That difference is not cosmetic – a higher correlation means a linear model can capture more of the underlying relationship with fewer parameters, which usually translates into better predictions on new data.

5.2 Splitting the data – the mock exam and the final exam

```
set.seed(101)
train_idx <- sample(seq_len(nrow(concrete_ml)),
                    size = floor(0.8 * nrow(concrete_ml)))
train_set <- concrete_ml[train_idx, ]
test_set  <- concrete_ml[-train_idx, ]

c(train_rows = nrow(train_set), test_rows = nrow(test_set))

## train_rows test_rows
##          96        24
```

How to read this output: With 120 rows and an 80/20 split, you should see `train_rows = 96` and `test_rows = 24`. The `set.seed()` makes the split reproducible – anyone who runs the same code gets the same partition, which is essential for comparing models fairly.

Randomness and reproducibility. Always set a seed before a random split. Without it, every run of your script gives a slightly different test set, and you can never be sure whether a change in performance came from a better model or just a luckier split.

5.3 Training a linear regression model

```
strength_model <- lm(strength ~ cement + water + log_age, data = train_set)
summary(strength_model)

##
## Call:
## lm(formula = strength ~ cement + water + log_age, data = train_set)
##
## Residuals:
```

```
##      Min      1Q  Median      3Q      Max
## -6.5910 -1.6364 -0.1673  1.2929  5.2719
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)  6.307838   2.110941   2.988  0.0036 **
## cement       0.080363   0.002615  30.729 <2e-16 ***
## water       -0.107898   0.009368 -11.518 <2e-16 ***
## log_age     12.048630   0.475373  25.346 <2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 2.359 on 92 degrees of freedom
## Multiple R-squared:  0.9492, Adjusted R-squared:  0.9475
## F-statistic: 572.9 on 3 and 92 DF,  p-value: < 2.2e-16
```

How to read the model summary – line by line:

- **Coefficients table.** Each row shows the estimated slope for one predictor.
 - (Intercept) – the predicted strength when all predictors are 0 (rarely physically meaningful, but mathematically required).
 - cement – the change in strength (MPa) for each extra 1 kg/m3 of cement, holding water and age fixed. You should see approximately +0.08.
 - water – the change for each extra litre of water; expected approximately -0.10 MPa per litre.
 - log_age – the change for each unit increase in log10(age); expected approximately +12 MPa.
- **Std. Error** – the uncertainty in each estimated slope. Smaller is better.
- **t value** – Estimate divided by Std. Error; large absolute values mean the coefficient is far from zero.
- **Pr(>|t|)** – the p-value. A tiny p-value (say less than 0.001) means “this relationship is very unlikely to be luck”.
- **Residual standard error** – the typical size of the model’s mistake on the training set (in MPa).
- **Multiple R-squared** – the fraction of variance in strength explained by the model. Closer to 1 is better.
- **F-statistic** – an overall test that *any* of the predictors matter. A tiny p-value here means the model is better than a bare average.

5.4 Predicting on the unseen test set

```
test_pred <- predict(strength_model, newdata = test_set)
head(data.frame(actual = test_set$strength, predicted = round(test_pred, 1)))
```

```
##      actual predicted
## 1      31.0      31.6
## 5      33.1      33.2
## 12     37.2      36.2
## 13     28.6      29.4
## 21     44.2      41.4
## 27     31.2      32.5
```

How to read this output: Each row shows the true strength of a cube the model has **never seen**, next to the model’s guess. The gaps between the two columns are the model’s errors. If the gaps are consistently

small and unbiased, the model is useful. If they are large or systematically off (say, always under-predicting high-strength mixes), there is a problem.

6 Evaluation Metrics for Regression (RMSE, MAE, R-squared)

6.1 What are we measuring?

Regression metrics answer two questions: “**How wrong are the predictions?**” and “**Is the model actually useful, or would a lazy guess do just as well?**”

Metric	Formula	Plain English	Watch out for
MAE	$\frac{1}{n} \sum y - \hat{y} $	“Average size of my mistake”	Treats all errors equally
RMSE	$\sqrt{\frac{1}{n} \sum (y - \hat{y})^2}$	“Average mistake, but big errors punished extra”	Sensitive to outliers (can be good!)
R2	$1 - \frac{SS_{res}}{SS_{tot}}$	“Share of variation I explained” (0 to 1)	High R2 alone can hide big errors

MAE vs RMSE – a tiny worked example: Suppose two models both make two errors. Model A’s errors are (4, 0); Model B’s errors are (2, 2).

- MAE for A = $(4 + 0)/2 = 2$. MAE for B = $(2 + 2)/2 = 2$. **Identical.**
- RMSE for A = square root of $((16 + 0)/2) = 2.83$. RMSE for B = square root of $((4 + 4)/2) = 2.00$. **Different!**

RMSE punishes Model A because it made one big blunder, while Model B made two small ones. For concrete strength – where a badly wrong guess is dangerous – RMSE is often the right choice.

6.2 Computing the metrics

```
# Reusable metric functions (we will use them again later!)
mae <- function(actual, predicted) mean(abs(actual - predicted))
rmse <- function(actual, predicted) sqrt(mean((actual - predicted)^2))
r_squared <- function(actual, predicted) {
  1 - sum((actual - predicted)^2) / sum((actual - mean(actual))^2)
}

knitr::kable(data.frame(
  Metric = c("MAE (MPa)", "RMSE (MPa)", "R-squared"),
  Value = c(
    round(mae(test_set$strength, test_pred), 2),
    round(rmse(test_set$strength, test_pred), 2),
    round(r_squared(test_set$strength, test_pred), 3)
  )
), caption = "Our model on the unseen test set")
```

Table 1: Our model on the unseen test set

Metric	Value
MAE (MPa)	1.890
RMSE (MPa)	2.320
R-squared	0.915

How to read this table:

- **MAE (MPa)** – the average absolute error. If it is 2.0, the model is typically off by about plus or minus 2 MPa.
- **RMSE (MPa)** – similar units to MAE, but always greater than or equal to MAE. If RMSE is much larger than MAE, the model is making a few large errors.
- **R-squared** – fraction of variation explained. 0.85 means the model captures 85% of the spread in strength; 1.0 would be perfect; 0 means no better than always guessing the mean.

6.3 Always compare with a lazy baseline

A model is only useful if it beats the dumbest possible strategy: **guess the average strength for every cube**.

```
baseline_pred <- rep(mean(train_set$strength), nrow(test_set))
c(
  "RMSE of lazy baseline" = round(rmse(test_set$strength, baseline_pred), 2),
  "RMSE of our model"     = round(rmse(test_set$strength, test_pred), 2)
)

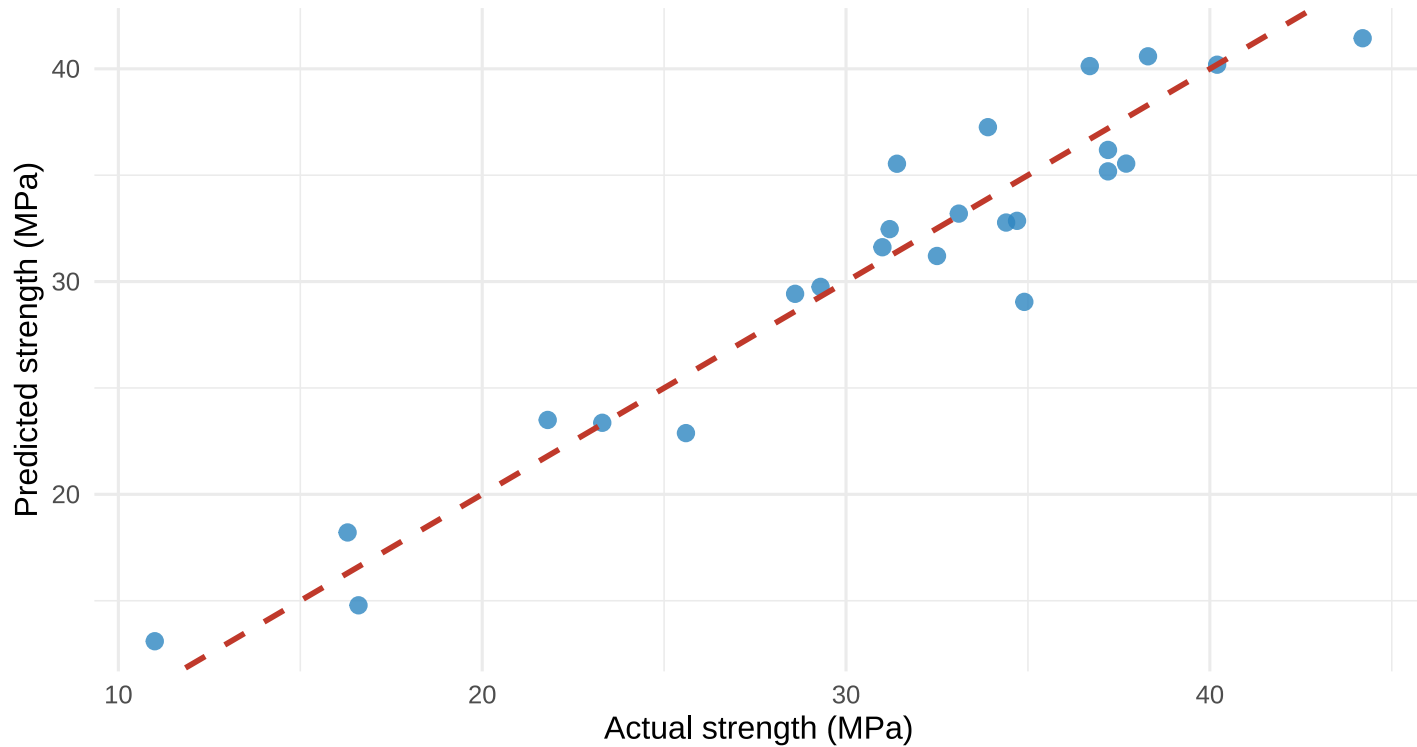
## RMSE of lazy baseline      RMSE of our model
##                          8.40                2.32
```

How to read this output: The lazy baseline simply predicts the training-mean for every cube. If your model's RMSE is **not meaningfully smaller** than the baseline, your features and model are contributing nothing – you would get the same result by taping a single number to the wall and calling it a predictor.

```
ggplot(
  data.frame(actual = test_set$strength, predicted = test_pred),
  aes(x = actual, y = predicted)
) +
  geom_point(color = "#2e86c1", size = 2.5, alpha = 0.8) +
  geom_abline(
    slope = 1, intercept = 0, linetype = "dashed",
    color = "#c0392b", linewidth = 1
  ) +
  labs(
    title = "Predicted vs actual concrete strength (test set)",
    subtitle = "Dashed line = perfect predictions; points hugging it = good model",
    x = "Actual strength (MPa)", y = "Predicted strength (MPa)"
  ) +
  theme_minimal(base_size = 12)
```

Predicted vs actual concrete strength (test set)

Dashed line = perfect predictions; points hugging it = good model



How to read this figure:

- Each blue point is one test-set cube. The horizontal axis is the **true** strength; the vertical axis is the **predicted** strength.
- The red dashed diagonal is the line $y = x$ – the “perfect prediction” line. A perfect model would put every point exactly on it.
- **Points above the line** = the model over-predicted (said the cube would be stronger than it was).
- **Points below the line** = the model under-predicted (said the cube would be weaker).
- **Horizontal band shape** across all actual values = the model is essentially predicting a constant – that is a failed model.
- **Sloped cloud hugging the diagonal** = the model has captured the real relationship.

Civil reading: “RMSE approximately 3 MPa” means predictions are typically within about plus or minus 3 MPa of the truth. If your design code requires plus or minus 2 MPa tolerance, this model is not good enough yet. Your options are: gather more data, engineer better features, or accept a different model class (Unit 4).

7 Evaluation Metrics for Classification (Accuracy, Precision, Recall, F1, ROC)

7.1 What is classification?

Classification predicts a **category** rather than a number. Example: “Will this catchment flood this monsoon: Yes or No?” Or “Is this slope stable or unstable?” Or “Is this weld pass or fail?”

We judge classifiers with a **confusion matrix** and its derived metrics: Accuracy, Precision, Recall, F1, and the ROC curve. Each metric highlights a different aspect of performance – and picking the right one is an engineering decision, not a mathematical one.

7.2 The flood-warning dataset (simulated for reproducibility)

```
set.seed(7)
n <- 200
rain <- runif(n, 10, 120)                # rainfall, mm
p_flood_true <- plogis((rain - 70) / 15)  # true flood probability
actual <- factor(ifelse(runif(n) < p_flood_true, "Flood", "No Flood"),
                 levels = c("Flood", "No Flood"))

p_model <- plogis((rain - 72) / 14 + rnorm(n, 0, 0.35)) # our model
predicted <- factor(ifelse(p_model >= 0.5, "Flood", "No Flood"),
                   levels = c("Flood", "No Flood"))
```

What we just created: 200 hypothetical monsoons. Each one has a rainfall amount, a true flood outcome (drawn from a logistic function of rain), and a model-predicted flood probability. The model’s version is slightly noisier and shifted from the truth – as real models are.

7.3 The Confusion Matrix – Four verbs of judgement

The confusion matrix is the foundation of all classification metrics. It is a 2x2 table that counts, for every case, whether the model got it right and *how* it got it right.

	Actually Flood	Actually No Flood
We warned (Flood)	TP – correct siren	FP – false alarm (costly evacuation)
We stayed quiet	FN – MISSED flood (dangerous)	TN – correct silence

Read it slowly. Each cell has a different engineering cost.

- **True Positive (TP)** – we warned, and it flooded. The siren worked. Success.
- **False Positive (FP)** – we warned, but nothing happened. False alarm. Evacuations are expensive, and repeated false alarms erode public trust in the warning system.
- **False Negative (FN)** – we stayed quiet, but it flooded. People were not warned. This is the dangerous failure – lives at risk.
- **True Negative (TN)** – we stayed quiet, and nothing happened. Correct silence.

```
cm <- table(Predicted = predicted, Actual = actual)
knitr::kable(cm, caption = "Confusion matrix for the flood-warning model")
```

Table 2: Confusion matrix for the flood-warning model

		Flood	No Flood
Flood	74	19	
No Flood	15	92	

How to read the confusion matrix:

- Each cell contains a count of cases. The four numbers must add up to the total number of test cases (200 here).
- The top-left cell is TP (we said Flood, it flooded). The bottom-right is TN. The top-right is FP, and the bottom-left is FN.
- **A good classifier** puts most of its counts on the diagonal (TP and TN) and leaves FP and FN small.
- Read the *rows* as “what the model said” and the *columns* as “what actually happened”. Different textbooks order them differently – always check the labels.

7.4 The Four Key Metrics

Each of these is a different way of combining TP, FP, FN, and TN into a single number.

$$\text{Accuracy} = \frac{TP + TN}{\text{All}} \quad \text{Precision} = \frac{TP}{TP + FP} \quad \text{Recall} = \frac{TP}{TP + FN} \quad F1 = \frac{2 \cdot P \cdot R}{P + R}$$

Civil meaning of each metric:

- **Accuracy** = “Out of every 100 predictions, how many were right?” – useful only when both classes are roughly equally common.
- **Precision** = “When the siren rang, how often was it real?” – a precision of 0.80 means 4 out of 5 warnings were genuine floods. Low precision means the boy who cried wolf.
- **Recall (Sensitivity)** = “Of all real floods, how many did we catch?” – recall of 0.67 means we caught 2 out of every 3 real floods. The other 1 slipped through silently. For life-safety systems, this is usually the most important number.
- **F1 Score** = the harmonic mean of precision and recall – a single number that balances the two. Useful when you want one headline figure and both false alarms and missed events matter.

```
TP <- cm["Flood", "Flood"]
FP <- cm["Flood", "No Flood"]
FN <- cm["No Flood", "Flood"]
TN <- cm["No Flood", "No Flood"]

P <- TP / (TP + FP)
R <- TP / (TP + FN)

knitr::kable(data.frame(
  Metric = c("Accuracy", "Precision", "Recall (Sensitivity)", "F1 Score"),
  Formula = c("(TP+TN)/All", "TP/(TP+FP)", "TP/(TP+FN)", "2PR/(P+R)"),
  Value = round(c((TP + TN) / sum(cm), P, R, 2 * P * R / (P + R)), 3)
))
```

Metric	Formula	Value
Accuracy	$(TP+TN)/All$	0.830
Precision	$TP/(TP+FP)$	0.796
Recall (Sensitivity)	$TP/(TP+FN)$	0.831
F1 Score	$2PR/(P+R)$	0.813

How to read this table:

- **Accuracy** close to 1.0 sounds impressive – but see the warning below.
- **Precision** near 1.0 means almost every warning was real. Near 0.5 means half the sirens were false alarms.
- **Recall** near 1.0 means we caught almost every flood. Near 0.3 means we missed 7 out of 10 real floods.
- **F1** is a combined score. A model with precision 1.0 and recall 0.2 has F1 approximately 0.33 – the F1 has already penalised the missed floods, even though precision looked perfect.

The accuracy trap: if floods happen only 1% of the time, a lazy model that ALWAYS says “No Flood” scores 99% accuracy and 0% recall – while missing every single flood. For rare, dangerous events, trust Recall and F1, **never accuracy alone**. This is the single most dangerous mistake in applied ML for civil engineering.

7.5 The Threshold Slider – Same model, different personalities

The 0.5 cut-off was our choice, not a law of nature. Move it and watch the trade-off between precision and recall.

```
thr_table <- do.call(rbind, lapply(c(0.30, 0.50, 0.70), function(t) {
  pred_t <- factor(ifelse(p_model >= t, "Flood", "No Flood"),
                    levels = c("Flood", "No Flood"))
  cm_t <- table(pred_t, actual)
  TP <- cm_t["Flood", "Flood"]
  FP <- cm_t["Flood", "No Flood"]
  FN <- cm_t["No Flood", "Flood"]
  TN <- cm_t["No Flood", "No Flood"]
  data.frame(
    Threshold = t,
    Accuracy = round((TP + TN) / sum(cm_t), 3),
    Precision = round(TP / (TP + FP), 3),
    Recall = round(TP / (TP + FN), 3)
  )
}))
knitr::kable(
  thr_table,
  row.names = FALSE,
  caption = "Lower threshold = more warnings = higher recall, more false alarms"
)
```

Table 3: Lower threshold = more warnings = higher recall, more false alarms

	Threshold	Accuracy	Precision	Recall
	0.3	0.775	0.690	0.899
	0.5	0.830	0.796	0.831
	0.7	0.840	0.890	0.730

How to read this table:

- **Threshold = 0.30** (trigger-happy): the model warns whenever it thinks there is even a 30% chance. Recall goes up (we catch more floods), but precision drops (more false alarms).
- **Threshold = 0.70** (cautious): the model only warns when it is quite confident. Precision goes up, but recall drops – some floods slip through unannounced.
- **Threshold = 0.50** is the default, not the best. Real engineering practice picks the threshold that minimises the *cost-weighted* damage – and in flood forecasting, the cost of a missed flood is far higher than a false alarm.

Engineering decision, not a maths decision: A downstream community that evacuates for every false alarm will lose trust in the warning system. But a community that misses a real flood loses lives. The threshold is where engineering, economics, and ethics meet.

7.6 The ROC Curve – Judging across ALL thresholds at once

The ROC curve plots **Recall (True Positive Rate)** against the **False Alarm Rate (False Positive Rate)** for every possible threshold. It shows the full range of possible trade-offs in a single picture. The **Area Under the Curve (AUC)** summarises that picture as one number: 1.0 = perfect, 0.5 = coin-flip guessing.

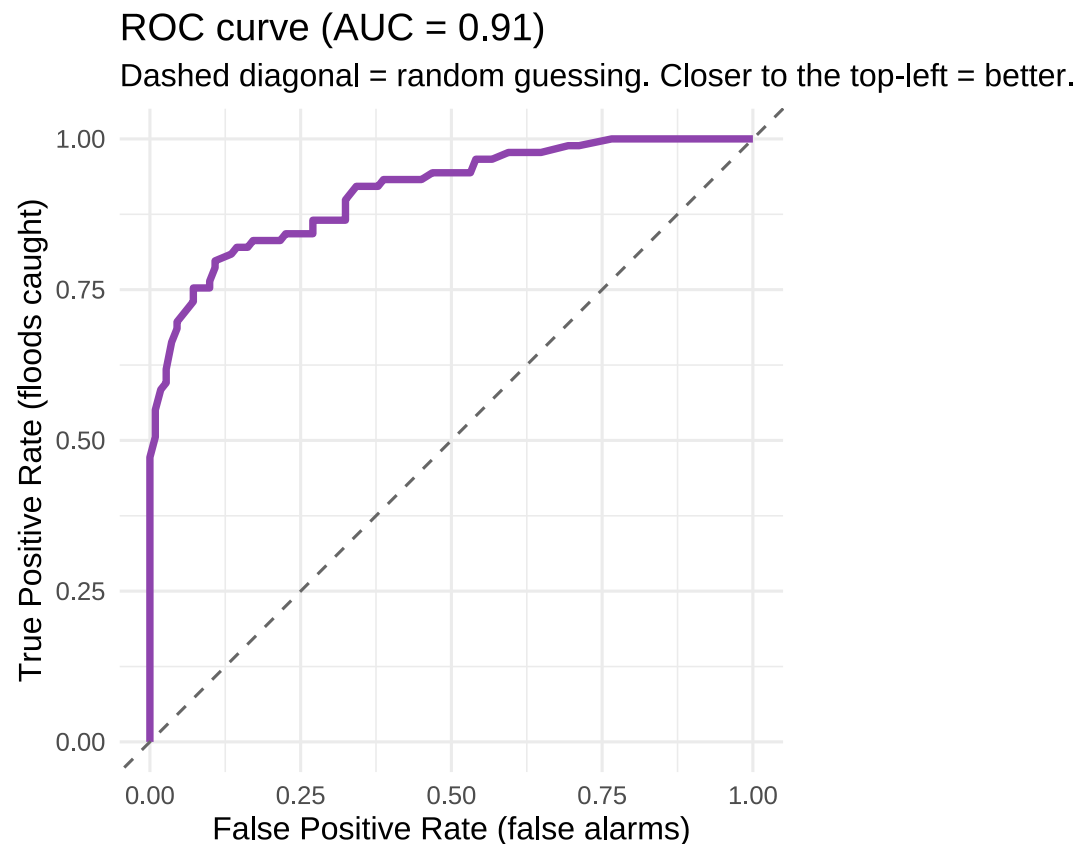
```
thresholds <- seq(0, 1, by = 0.01)
tpr <- sapply(thresholds, function(t) {
  sum(p_model >= t & actual == "Flood") / sum(actual == "Flood")
})
fpr <- sapply(thresholds, function(t) {
  sum(p_model >= t & actual == "No Flood") / sum(actual == "No Flood")
})

roc_df <- data.frame(fpr, tpr)[order(fpr, tpr), ]
auc <- sum(diff(roc_df$fpr) *
           (head(roc_df$tpr, -1) + tail(roc_df$tpr, -1)) / 2) # trapezoid
c(AUC = round(auc, 3))
```

```
## AUC
## 0.911
```

```
ggplot(roc_df, aes(x = fpr, y = tpr)) +
  geom_line(color = "#8e44ad", linewidth = 1.3) +
  geom_abline(slope = 1, intercept = 0, linetype = "dashed",
             color = "grey40") +
  coord_equal(xlim = c(0, 1), ylim = c(0, 1)) +
```

```
labs(
  title = paste0("ROC curve (AUC = ", round(auc, 2), ")"),
  subtitle = "Dashed diagonal = random guessing. Closer to the top-left = better.",
  x = "False Positive Rate (false alarms)",
  y = "True Positive Rate (floods caught)"
) +
theme_minimal(base_size = 12)
```



How to read the ROC curve:

- **Horizontal axis (FPR)** = “of all the quiet monsoons, what fraction did we cry wolf on?” Lower is better.
- **Vertical axis (TPR)** = “of all the real floods, what fraction did we catch?” Higher is better.
- **Purple curve** = the model’s performance at every threshold. Each point on the curve corresponds to a specific cut-off (0.30, 0.50, 0.70, etc.).
- **Dashed diagonal** = the line of a model that has learned nothing. If our curve lies on it, our predictions are no better than flipping a coin.
- **AUC (area under the curve)** = a single number summarising overall quality. 0.5 = useless, 1.0 = perfect, and typically 0.7–0.9 is considered good in practice.
- The top-left corner (0, 1) is the ideal – catch every flood and never raise a false alarm. Real models compromise along the curve.

Note: we built the ROC by hand so you understand the mechanics. In practice, the `pROC` or `ROCR` packages compute this in one line – and Unit 4’s logistic regression will produce these probabilities directly, so the pipeline connects cleanly.

8 Statistical Challenges – Overfitting, Underfitting and the Bias-Variance Tradeoff

8.1 What is the Goldilocks problem?

In machine learning, a model can be **too simple**, **too complicated**, or **just right**. Too simple and it misses the pattern. Too complicated and it memorises noise. Just right and it generalises to new data.

The student analogy:

- **Underfitting** = a student who did not study at all. Fails the mock exam AND the final. Poor everywhere.
- **Overfitting** = a student who memorised the mock exam answers word-for-word. Aces the mock, fails the final because the questions changed.
- **Good fit** = a student who understood the concepts. Good on both exams, and can answer questions they have never seen before.

8.2 The experiment: seasonal groundwater level

Groundwater follows a seasonal wave. We fit polynomials of degree 1 (straight line), degree 4 (smooth curve), and degree 15 (a wiggly noodle) and compare training vs. test RMSE.

```
set.seed(100)
n_fit <- 60
x_all <- runif(n_fit, 0, 12)           # month of year
y_all <- 8 + 3 * sin(2 * pi * x_all / 12) +
  rnorm(n_fit, 0, 0.8)                # water table (m)

split_idx <- sample(seq_len(n_fit), size = 0.7 * n_fit)
d_train <- data.frame(x = x_all[split_idx], y = y_all[split_idx])
d_test  <- data.frame(x = x_all[-split_idx], y = y_all[-split_idx])

degrees <- c(1, 4, 15)
results <- do.call(rbind, lapply(degrees, function(d) {
  m <- lm(y ~ poly(x, d), data = d_train)
  data.frame(
    Degree      = d,
    Train_RMSE = round(rmse(d_train$y, predict(m)), 3),
    Test_RMSE  = round(rmse(d_test$y,  predict(m, newdata = d_test)), 3)
  )
}))
results$Verdict <- c("Too simple: UNDERFITS",
  "Good balance",
  "Memorises noise: OVERFITS")
results
```

##	Degree	Train_RMSE	Test_RMSE	Verdict
## 1	1	1.656	0.965	Too simple: UNDERFITS
## 2	4	0.896	0.929	Good balance
## 3	15	0.755	0.960	Memorises noise: OVERFITS

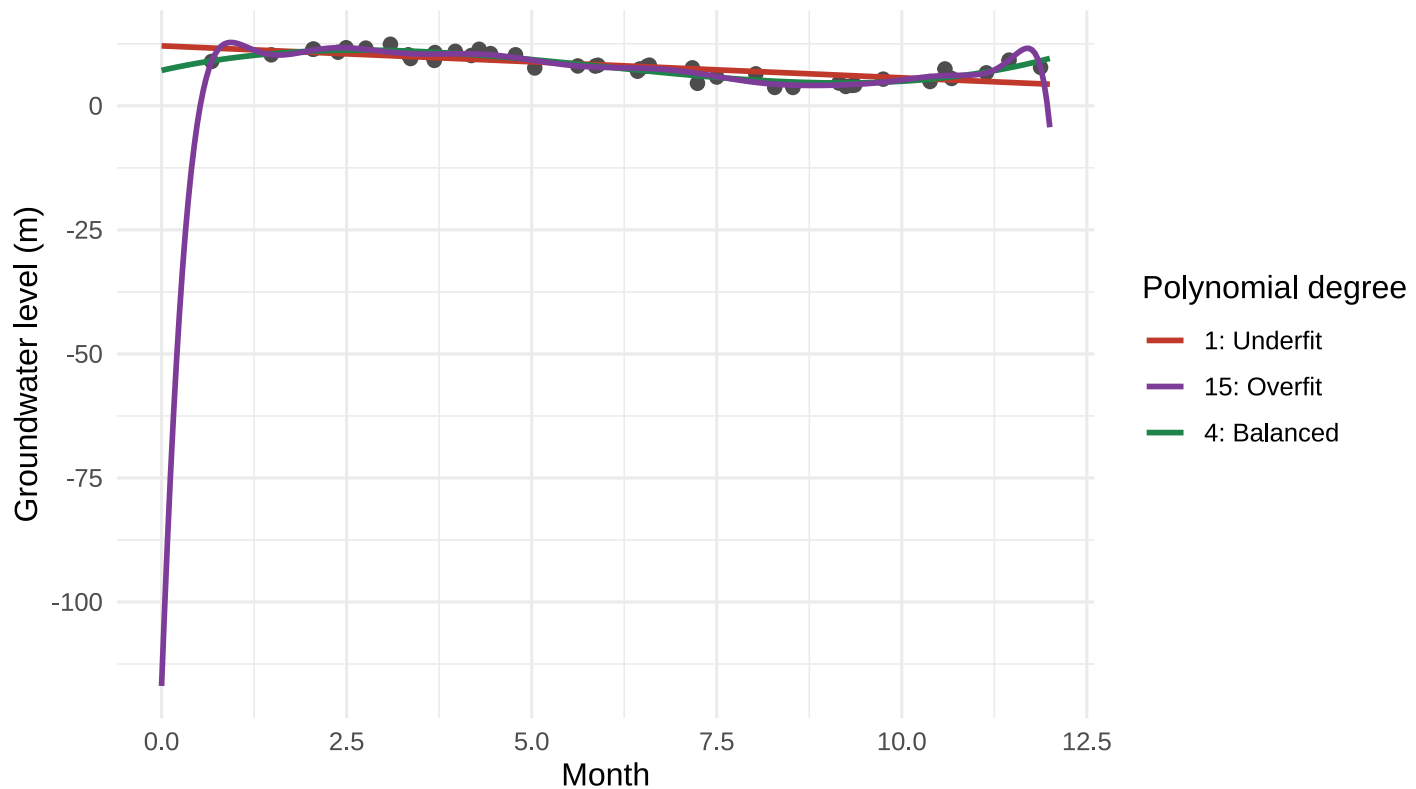
How to read this table – this is the most important table in the whole unit:

- **Degree 1 (straight line).** Train RMSE and Test RMSE are both moderate – the model cannot bend enough to follow the seasonal wave. It is **underfitting**.
- **Degree 4 (smooth curve).** Both train and test RMSE drop to low values. The curve matches the shape of the true seasonal wave. This is the **sweet spot**.
- **Degree 15 (wiggly noodle).** Train RMSE is the **lowest** – it twists and turns to touch every training point. But Test RMSE is **much worse**. The model memorised the noise. It is **overfitting**.
- **The verdict column** names each row's problem. Trust the Test RMSE, not the Train RMSE, when choosing a model.

```
grid <- data.frame(x = seq(0, 12, length.out = 300))
grid$deg1 <- predict(lm(y ~ poly(x, 1), d_train), newdata = grid)
grid$deg4 <- predict(lm(y ~ poly(x, 4), d_train), newdata = grid)
grid$deg15 <- predict(lm(y ~ poly(x, 15), d_train), newdata = grid)

ggplot() +
  geom_point(data = d_train, aes(x, y), color = "grey30", size = 2) +
  geom_line(data = grid, aes(x, deg1, color = "1: Underfit"), linewidth = 1) +
  geom_line(data = grid, aes(x, deg4, color = "4: Balanced"), linewidth = 1) +
  geom_line(data = grid, aes(x, deg15, color = "15: Overfit"), linewidth = 1) +
  scale_color_manual(values = c(
    "1: Underfit" = "#c0392b",
    "4: Balanced" = "#1e8449",
    "15: Overfit" = "#7d3c98"
  ), name = "Polynomial degree") +
  labs(
    title = "Same data, three models -- which would you trust on a new site?",
    x = "Month", y = "Groundwater level (m)"
  ) +
  theme_minimal(base_size = 12)
```

Same data, three models -- which would you trust on a new site?



How to read this figure:

- **Black dots** = the training data (groundwater measurements over 12 months). Notice they follow a wavy seasonal pattern with visible scatter.
- **Red line (degree 1)** = a straight line through a wavy pattern. It systematically misses the peaks and troughs – this is **underfitting**.
- **Green line (degree 4)** = a smooth curve that follows the seasonal wave. It captures the peaks and troughs without chasing individual dots. This is the **good fit**.
- **Purple line (degree 15)** = it twists through every dot, diving between them and oscillating wildly outside them. On a new site, its predictions would be nonsense – this is **overfitting**.
- Ask yourself: “If I had a new groundwater reading at month 9, which line would give the best prediction?” The green one – every time.

8.3 The Bias-Variance Tradeoff

Total expected error decomposes as:

$$\text{Total Error} = \text{Bias}^2 + \text{Variance} + \text{Irreducible Noise}$$

Think of a **dartboard**:

- **Bias** = systematically aiming wrong. All darts cluster in one place, but it is not the bullseye.
- **Variance** = shaky hands. Darts scatter all over the board.
- **Irreducible noise** = the natural jitter of the universe. No model can eliminate it.

Model	Bias	Variance	Behaviour
Degree 1 (straight line on a wave)	High	Low	Underfits
Degree 4 (smooth curve)	Medium	Medium	Just right
Degree 15 (noodle)	Low	High	Overfits

Why the trade-off exists: a very flexible model can fit any pattern, including the noise – so its bias is low, but it responds differently to every random training set, so its variance is high. A very rigid model does the opposite.

Civil analogy: Imagine two surveyors measuring the elevation of a road.

- **Underfit surveyor** measures every 500 m and just draws straight lines between the points – misses every dip and crest.
- **Overfit surveyor** measures every 5 cm and models every pebble – very accurate on this specific road, useless on any other road.
- **Good surveyor** measures every 20 m and captures the true road profile – usable on the next road too.

How engineers fight the trade-off: more data, simpler models, fewer features, regularisation (Ridge/Lasso) – and **cross-validation** to *find* the sweet spot. Which brings us to the final section.

9 Cross-Validation Techniques – Many Mock Exams, Not One

9.1 What is cross-validation?

A single train/test split can be lucky or unlucky. If your test set happens to contain only easy cases, the model looks great; if it happens to contain a few weird cases, the model looks terrible. Neither number reflects the model's true quality.

K-fold cross-validation fixes this. The training data is split into **k folds** (usually 5 or 10). We train on k-1 folds and test on the remaining fold, then rotate. Every data point gets examined exactly once, and we average the k scores.

```
fold_demo <- matrix("Train", nrow = 5, ncol = 5)
diag(fold_demo) <- "TEST"
knitr::kable(fold_demo,
              col.names = paste0("Fold ", 1:5),
              caption = "5-fold CV: in each round, one fold is the test set (rotating)")
```

Table 4: 5-fold CV: in each round, one fold is the test set (rotating)

Fold 1	Fold 2	Fold 3	Fold 4	Fold 5
TEST	Train	Train	Train	Train
Train	TEST	Train	Train	Train
Train	Train	TEST	Train	Train
Train	Train	Train	TEST	Train
Train	Train	Train	Train	TEST

How to read this table:

- Each **row** is one round of cross-validation.
- Each **column** is one fold of the data.
- In round 1, Fold 1 is the test set and Folds 2–5 are used for training. In round 2, Fold 2 is the test set, and so on.
- By the time we finish 5 rounds, **every fold has been tested exactly once**, and every fold has been trained on 4 times.
- You end up with **5 different test scores** – and their average is a much more reliable estimate of the model's true performance than any single split.

9.2 Cross-validation in practice

Let us use 5-fold CV to compare two models: one with the engineered `log_age` feature and one without.

```
set.seed(5)
k <- 5
folds <- sample(rep(1:k, length.out = nrow(train_set)))

cv_results <- data.frame(
  fold      = 1:k,
  rmse_with_log_age = NA,
  rmse_without_log_age = NA
)

for (i in 1:k) {
  cv_train <- train_set[folds != i, ]
  cv_test  <- train_set[folds == i, ]

  m_full  <- lm(strength ~ cement + water + log_age, data = cv_train)
  m_simple <- lm(strength ~ cement + water,          data = cv_train)

  cv_results$rmse_with_log_age[i] <-
    rmse(cv_test$strength, predict(m_full, newdata = cv_test))
  cv_results$rmse_without_log_age[i] <-
    rmse(cv_test$strength, predict(m_simple, newdata = cv_test))
}
cv_results
```

```
##   fold rmse_with_log_age rmse_without_log_age
## 1    1          2.465418          5.831858
## 2    2          2.681817          6.701626
## 3    3          2.485740          7.733147
## 4    4          2.509193          6.283093
## 5    5          1.657015          6.748487
```

How to read this table:

- Each row is one fold's evaluation. The two RMSE columns are the two competing models.
- No single fold is definitive – they all differ slightly because each fold contains different rows.
- Look at the **consistency**. If the `with_log_age` column is consistently lower than the `without_log_age` column, the feature genuinely helps. If they swap back and forth, the difference is noise.

```
knitr::kable(data.frame(
  Model      = c("With log_age feature", "Without log_age"),
  Mean_CV_RMSE = round(colMeans(cv_results[, 2:3]), 2),
  SD_CV_RMSE  = round(apply(cv_results[, 2:3], 2, sd), 2)
), caption = "Average of 5 exams is far more trustworthy than one exam")
```

Table 5: Average of 5 exams is far more trustworthy than one exam

	Model	Mean_CV_RMSE	SD_CV_RMSE	
rmse_with_log_age	With log_age feature	2.36		0.40
rmse_without_log_age	Without log_age	6.66		0.71

How to read this summary:

- **Mean_CV_RMSE** is the average test error across the 5 folds – this is the honest performance estimate.
- **SD_CV_RMSE** is the standard deviation across folds – this tells you how stable the model is. A small SD means every fold gave a similar answer; a large SD means performance depends heavily on which rows happened to be in the test fold.
- The winning model is the one with the lower mean RMSE – *provided* the SD is not huge.

Civil reading: CV confirms that `log_age` genuinely helps – this is the honest way to justify design decisions. It is exactly like a load test that averages several specimen cubes instead of trusting one. A single cube might be strong or weak by chance; the average is what you design to.

Variants you should recognise:

- **LOOCV (Leave-One-Out CV):** $k = n$. Every single row takes a turn as the test set. Very accurate but slow – because you train n models.
- **Repeated k-fold:** run the whole k -fold procedure several times with different random fold assignments, then average across repetitions. Extra stability at extra cost.
- **Stratified k-fold:** ensures each fold has the same class proportions as the full dataset. Essential for imbalanced classification (e.g., rare floods).
- **Time-series CV:** for temporal data, folds must respect time order – you never train on future data to predict the past.
- **Golden rule:** run CV *inside the training data only* to choose models; touch the held-out test set **once**, at the very end. The test set is your final exam, not your homework.

10 One-Page Summary

Concept	One-line takeaway
ML workflow	Ask, Collect, Prepare, Split, Train, Evaluate, Deploy
Cleaning	No NA, no duplicates, no impossible values before modelling
Min-Max / Z-score	Fair comparison when columns have different units or spreads
One-hot encoding	Words to 1/0 columns (for categories with no natural order)
Feature engineering	Rainfall intensity, runoff C, NDVI anomaly, $\log(\text{age}) = \text{raw data made meaningful}$

Concept	One-line takeaway
Train/test split	Mock exam vs. final exam – never leak
MAE / RMSE / R2	Avg error / avg error punishing blunders / percent of variation explained
Precision / Recall	Alarm accuracy / floods caught – rare dangers need recall
F1 / ROC-AUC	Balance of P and R / quality across all thresholds
Overfitting	Memorises noise; train error down, test error up
Bias-Variance	Too simple vs. too wiggly – seek the sweet spot
K-fold CV	Average of k exams; the honest model selector

11 Glossary

- **Feature** – an input column the model learns from (cement, rainfall intensity).
- **Label / Target** – what we predict (strength, Flood / No Flood).
- **Training set** – data the model learns from. **Test set** – data used for honest grading.
- **Imputation** – filling missing values (e.g., with the median).
- **Normalization / Standardization** – rescaling features to comparable ranges.
- **One-hot encoding** – one 1/0 column per category.
- **Confusion matrix** – 2x2 table of TP, FP, FN, TN.
- **ROC curve / AUC** – performance across all thresholds / area under it.
- **Bias** – systematic error from a too-simple model. **Variance** – instability of a too-complex model.
- **Cross-validation** – rotating train/test rounds averaged for reliability.
- **Data leakage** – any accidental leakage of test information into training. Always check.

12 Exit Ticket

- Q1.** Normalise cement = 350 kg/m³ given min = 200 and max = 500.
- Q2.** A storm drops 48 mm in 1.5 hours. Compute the rainfall intensity.
- Q3.** Using the Rational Method, find Q for C = 0.75, i = 40 mm/hr, A = 10 ha.
- Q4.** A flood model gives TP = 40, FP = 10, FN = 20, TN = 130. Find Accuracy, Precision, Recall, and F1.
- Q5.** Why can 99% accuracy be a disaster for flood prediction?
- Q6.** In a 5-fold cross-validation, why do we average across folds instead of trusting one?
- Q7.** A model has train RMSE = 0.8 and test RMSE = 6.4. Is it underfitting or overfitting? What is one fix?

Answers

A1. $x' = (350 - 200)/(500 - 200) = 0.5$

A2. $I = 48/1.5 = 32 \text{ mm/hr.}$

A3. $Q = 0.75 \times 40 \times 10/360 \approx 8.33 \text{ m}^3/\text{s.}$

A4. Accuracy = $170/200 = \mathbf{0.85}$; Precision = $40/50 = \mathbf{0.80}$; Recall = $40/60 = \mathbf{0.667}$; F1 = $2(0.8)(0.667)/(0.8+0.667) = \mathbf{0.727}$.

A5. If floods are 1% of cases, “always say No Flood” scores 99% accuracy but **Recall = 0** – it misses every real flood, which is exactly the dangerous failure mode. Use Recall / F1 for rare, hazardous events.

A6. A single fold might happen to contain only easy (or only hard) cases. Averaging across all folds gives a more stable, less luck-dependent estimate of true performance.

A7. Overfitting – training error is much smaller than test error. Fixes: simplify the model, gather more data, remove noisy features, or apply regularisation.

13 Further Reading

1. James, G., Witten, D., Hastie, T., and Tibshirani, R. (2021). *An Introduction to Statistical Learning* (2nd ed.) – Chapters 2 and 5 (resampling / CV). Free at statlearning.com.
2. Wickham, H., Cetinkaya-Rundel, M., and Grolemund, G. (2023). *R for Data Science* (2nd ed.) – data cleaning and transformation. Free at r4ds.hadley.nz.
3. Musoko, C. (2024). *Explainable Machine Learning for Geospatial Data Analysis* – feature engineering for spatial data.
4. Course notes 24CV201T, Units 3 and 4 (next unit: real algorithms – logistic regression, trees, KNN – evaluated with exactly these metrics).